



WEEK 4

FULL-STACK APPLICATION DEVELOPMENT

Backend Fundamentals & API Concepts

Instructor: Diloan Matini

TOPICS THIS WEEK

Backend Servers

Node.js

Express.js

APIs

Routes

Request & Response

MySQL Connection

REST Client

What We Learned Last Week

Databases

- Tables
- Rows
- Columns
- SQL

CRUD Operations

- Create
- Read
- Update
- Delete

IMPORTANT QUESTION

How does a website communicate with a database?

Backend Fundamentals

A backend is: The part of an application that runs on a server and processes data and business logic.

RESPONSIBILITIES

↓ Receive requests

⚙️ Process data

🗄️ Access databases

✅ Apply business rules

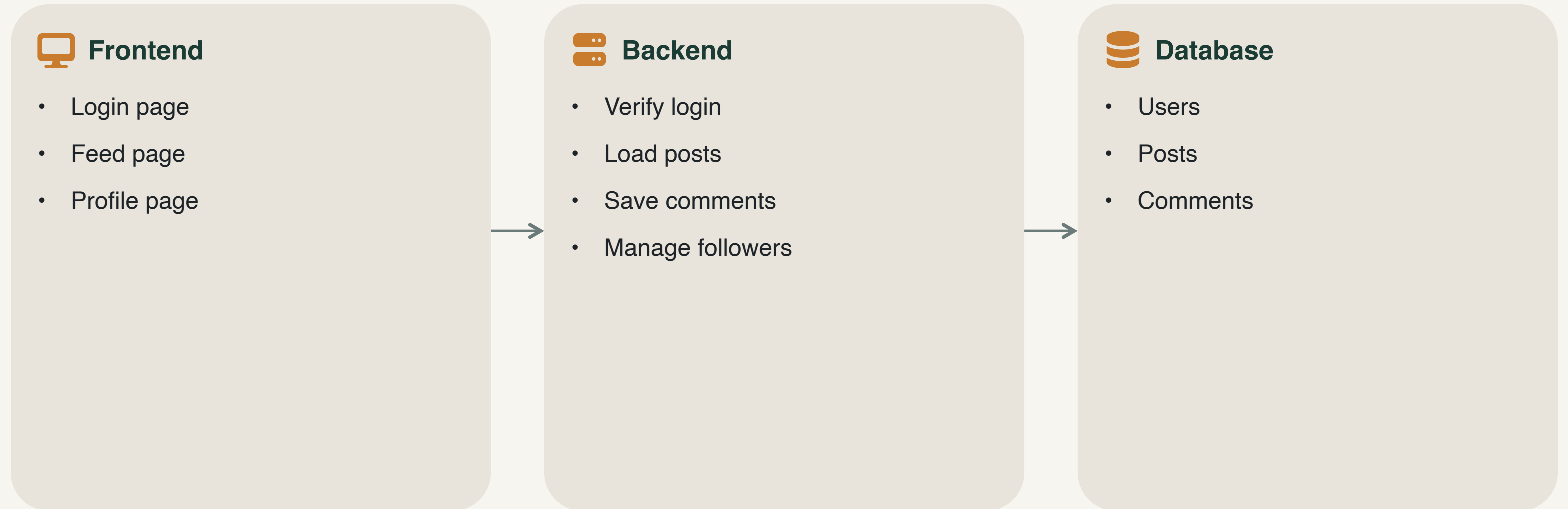
➡️ Send responses

Frontend vs Backend

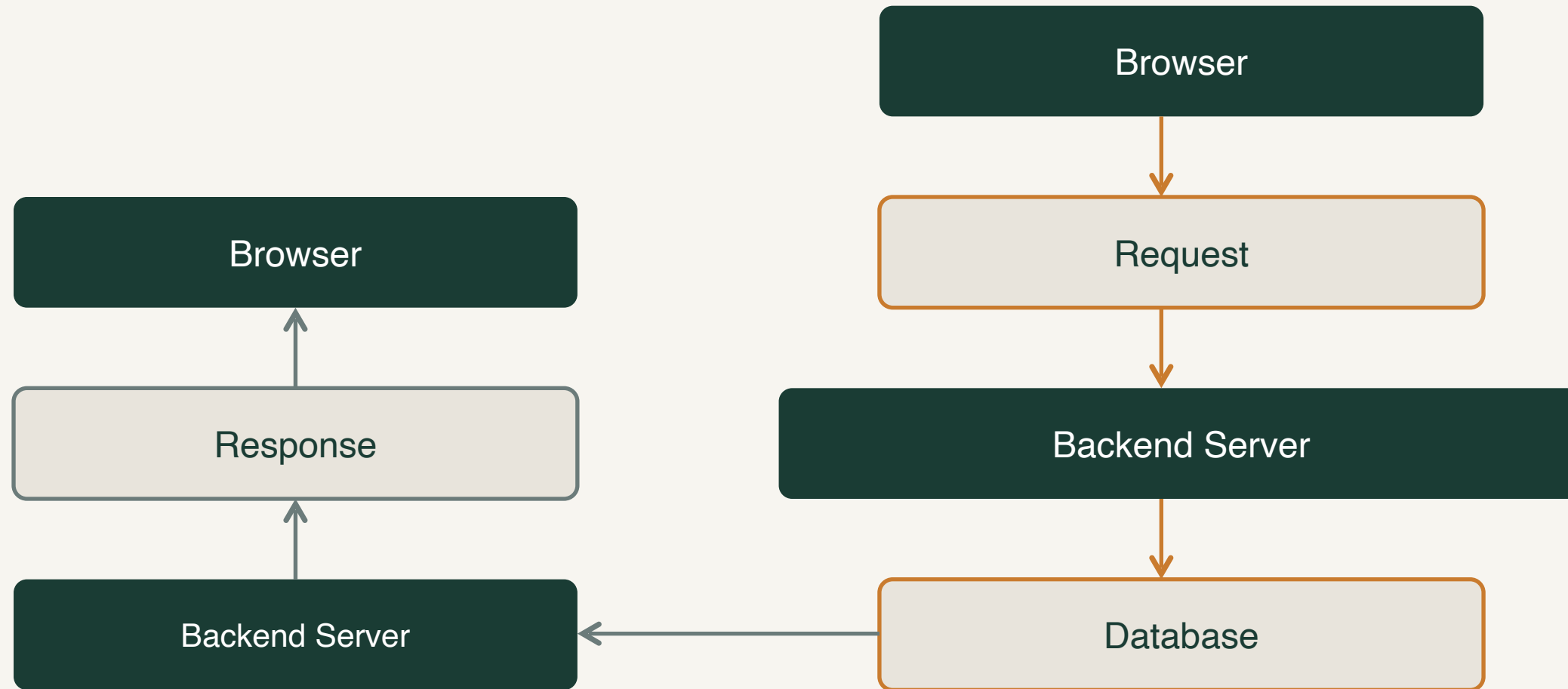
Aspect	Frontend	Backend
Visibility	What users see	Hidden logic
Runtime	Runs in browser	Runs on server
Technologies	HTML, CSS, JS	Node.js, Express
Focus	User interface	Data processing

Frontend + Backend = Full Stack Application

Instagram Example



Application Flow



The Backend Is The Middle Layer

API Fundamentals

API =

Application Programming Interface

SIMPLE DEFINITION

A way for applications to communicate with each other.

EXAMPLES

 Google Maps API

 Weather API

 OpenAI API

 Payment APIs

Request & Response

REQUEST

Sent by the client:

```
GET /users
```

RESPONSE

Returned by the server:

```
[  
  {  
    "id": 1,  
    "name": "Ali"  
  }  
]
```

DAY 2

Express.js Fundamentals

Node.js · Express.js · Creating Servers

Node.js

DEFINITION

Node.js allows **JavaScript** to run outside the browser.

BEFORE NODE.JS

JavaScript only worked inside browsers.

WITH NODE.JS

JavaScript can:

- Create servers
- Access databases
- Read files
- Build APIs

Why Use Node.js?

ADVANTAGES

 Fast

 Lightweight

 Popular

 Large community

 Uses JavaScript everywhere

ONE LANGUAGE

Frontend + Backend = JavaScript

Express.js

Express is a Node.js framework used to build web servers and APIs.

WHY EXPRESS?

 Simple

 Fast

 Popular

 Easy routing

 Large ecosystem

Create Backend Project

```
$ mkdir backend
```

```
$ cd backend
```

```
$ npm init -y
```

EXPLANATION

mkdir

Create folder

cd

Enter folder

npm init

Create package.json

Install Express.js

INSTALL

```
$ npm install express
```

RESULT

A new folder appears:

node_modules

Containing all Express dependencies.

Creating a Server

```
const express = require('express');  
const app = express();  
app.listen(3000);
```

WHAT HAPPENS?

A server starts on **port 3000**.

Returning Data

```
app.get('/', (req, res) => {  
  res.send('Hello World');  
});
```

RESULT

Visiting the URL shows:

Hello World

Start Server

COMMAND

```
$ node app.js
```

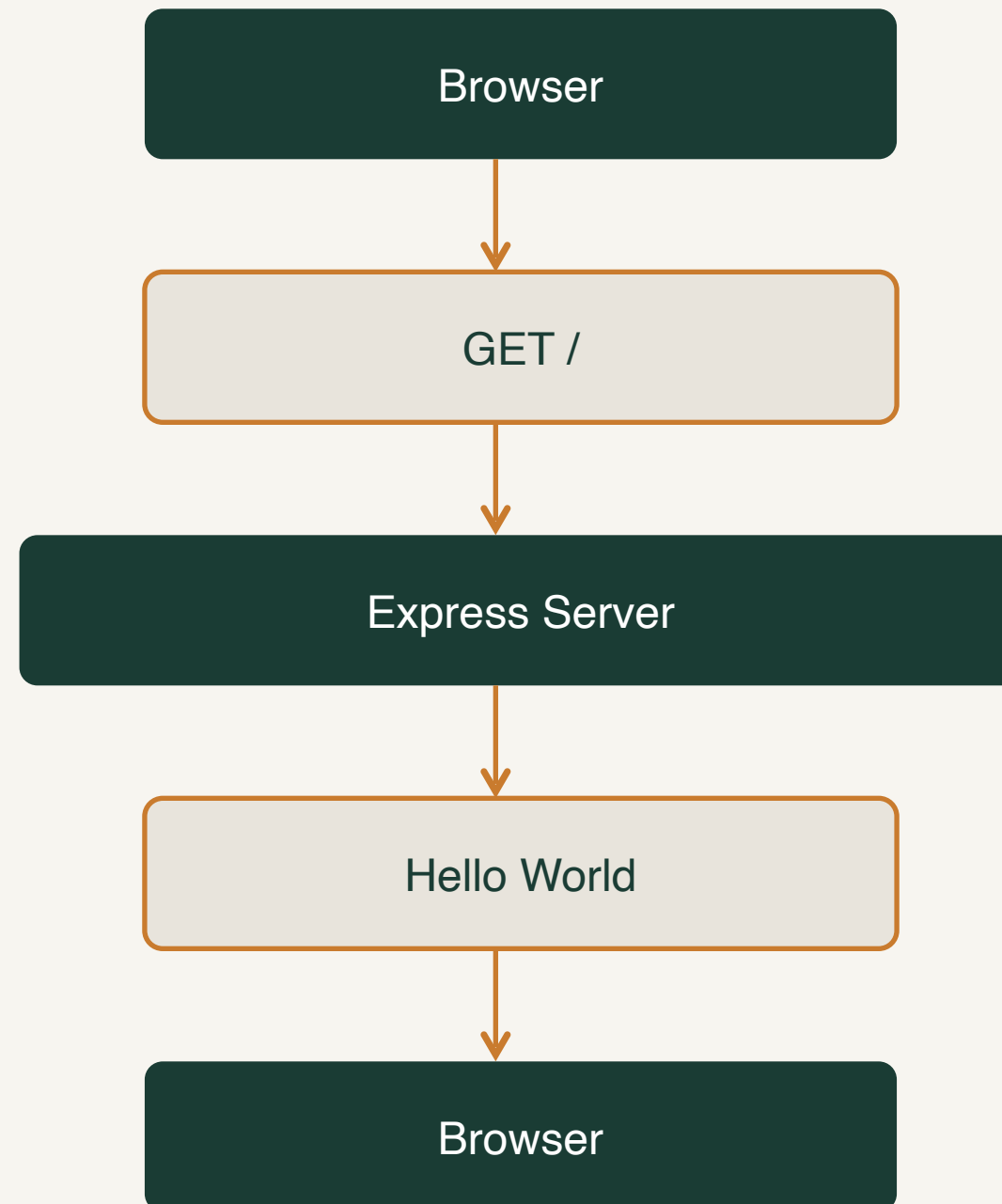
BROWSER URL

```
http://localhost:3000
```

IMPORTANT

localhost = Your own computer

What Happens Internally?



DAY 3

Routes & APIs

HTTP Methods · JSON · REST Client

Routes

Routes are **URLs handled by the backend.**

EXAMPLES

/

/users

/products

/orders

Basic Route Example

```
app.get('/users', (req, res) => {  
  res.send('Users');  
});
```

URL

```
http://localhost:3000/users
```

Multiple Endpoints

```
app.get('/')  
app.get('/users')  
app.get('/products')  
app.get('/orders')
```

One Server

Many Routes

HTTP Methods

Method	CRUD	Purpose
GET	Read	Retrieve data
POST	Create	Add data
PUT	Update	Update data
DELETE	Delete	Remove data

GET Example

```
app.get('/students', (req, res) => {  
  res.send('Student List');  
});
```

PURPOSE

Retrieve information from server.

JSON Responses

```
res.json([
  {
    id: 1,
    name: 'Ali'
  }
]);
```

WHY JSON?

-  Lightweight
-  Human readable
-  Standard API format

REST Client Extension

INSTALL FOR VISUAL STUDIO CODE

REST Client

EXAMPLE

GET http://localhost:3000/users

Send Request → View Response

DAY 4

Database Connection

MySQL · Queries · Full Application Flow

Why Databases Matter

WITHOUT DATABASE

Data disappears

WITH DATABASE

Data is stored **permanently**

EXAMPLES

 Users

 Products

 Orders

 Messages

MySQL Package

INSTALL

```
$ npm install mysql2
```

PURPOSE

Allow **Node.js** to communicate with **MySQL**.

Database Connection

```
const mysql = require('mysql2');  
const connection = mysql.createConnection({  
  host: 'localhost',  
  user: 'root',  
  password: '',  
  database: 'school_system'  
});
```

RESULT

Creates Database Connection

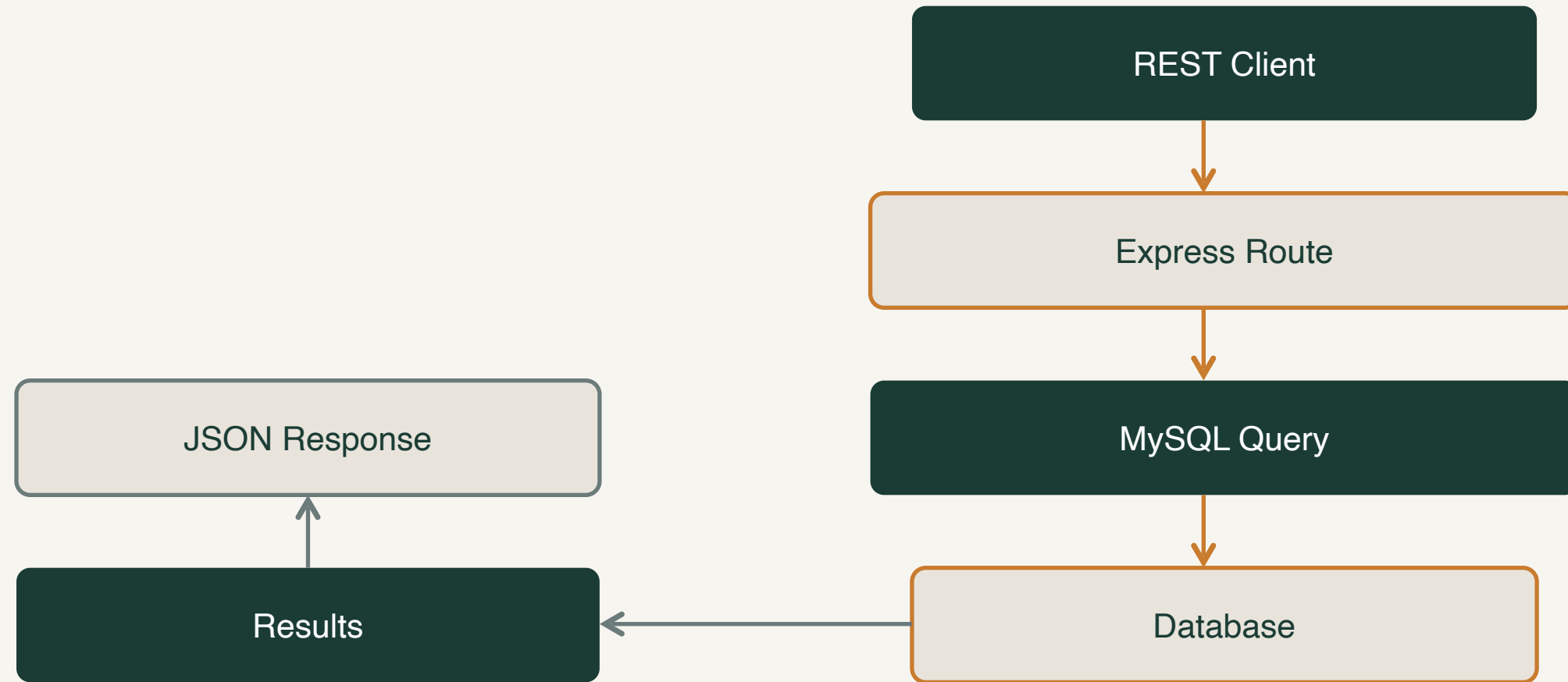
Execute Query

```
connection.query(  
  'SELECT * FROM students',  
  (err, results) => {  
    console.log(results);  
  }  
);
```

RESULT

Retrieve data from database.

Complete Backend Flow



This Is How Modern Applications Work

What We Learned

BACKEND CONCEPTS

- Backend servers
- APIs
- Requests
- Responses

EXPRESS.JS

- Create projects
- Create servers
- Create routes

DATABASE INTEGRATION

- Connect MySQL
- Execute queries

TESTING

- REST Client

NEXT WEEK

Frontend Development Fundamentals

TOPICS

React

State

Components

User Interfaces

Props

Backend + Frontend = Real Applications